Security & Governance: MCP Relay

Status: Draft v0.1
Last Updated: 2026-04-09

Contents

1. Purpose
2. What MCP Relay Is
3. External Service Integrations
4. Data Flow & Data Handling
5. Authentication & Authorisation
6. Credential Management
7. Adding New Tool Integrations
8. Audit & Logging
9. Risk Summary

---

1. Purpose

MCP Relay requires OAuth client registration. As part of this requirement, this document provides the security and governance detail needed to evaluate the application.

MCP Relay:

- Integrates with corporate IdP via OAuth 2.1 for user authentication
- Connects to external services (Jira, AWS S3, and future integrations) using centrally managed API credentials
- Acts as a controlled gateway — all external service access flows through a single, auditable server

The sections below cover what the server does, what data it handles, how credentials are managed, and how new integrations are governed.

For full system design, see the Architecture document. For infrastructure requirements, see the Infrastructure document.

---

2. What MCP Relay Is

MCP Relay is a centralised server implementing the Model Context Protocol (MCP). It exposes a curated set of AI tools — Jira issue management, brand guideline lookups, and future integrations — to MCP-compatible clients (Claude Code, Claude Desktop, Cowork).

What it is:

- A thin proxy layer that validates requests, calls external APIs, and returns formatted results

- A single point of credential management — individual users never handle API tokens directly
- A server that enforces input validation, rate limiting, and structured logging on every tool invocation

What it is not:

- It does not store user data beyond transient request/response cycles
- It does not make autonomous decisions — every action is initiated by a user through an MCP client
- It does not bypass existing service permissions — if a user's Jira account lacks access to a board, MCP Relay cannot circumvent that

---

3. External Service Integrations

Service	Purpose	Permissions Required	Data Accessed
Jira Cloud	Issue CRUD — get, create, search, update	Read/write issues via REST API v3	Issue fields: key, summary, status, assignee, description
AWS S3	Fetch per-project brand guidelines	Read-only access to a single S3 bucket	JSON configuration files (<1MB each)
Azure DevOps	Pipeline status, work items	Read-only (TBC)	TBC
SonarQube / Snyk	Code quality, dependency scanning	Read-only (TBC)	TBC
Confluence	Documentation search	Read-only (TBC)	TBC
Nanobanana/Gemini	AI image generation	API call (TBC)	TBC

Jira, S3, and Confluence are used by the team today. Azure DevOps, Nanobanana, and Gemini have been used in prior prototypes. SonarQube / Snyk is exploratory — no team member has requested it. The list above represents candidate integrations, not confirmed commitments. Each new integration follows the governance process defined in Section 7.

---

4. Data Flow & Data Handling

Request Lifecycle

```
User (authenticated via SSO)
    │
    ▼
MCP Client (Claude Code, Cowork, etc.)
    │
    ▼ HTTPS (TLS 1.2+)
MCP Relay Server
    │
    ├── 1. Authenticate user (validate OAuth 2.1 token against IdP)
    ├── 2. Validate input (Zod schema — reject malformed requests)
    ├── 3. Rate limit check (per-tool, per-user limits)
    ├── 4. Call external API (using server-held credentials)
    ├── 5. Validate response (Zod schema — catch malformed upstream data)
    ├── 6. Log invocation (structured JSON — who, what, when, outcome)
    └── 7. Return formatted result to MCP client
```

What data flows where

Data	Source	Destination	Stored?	Retention
User identity (SSO token)	Corporate IdP	MCP Relay	No (transient)	Request lifetime
Tool input parameters	MCP client	MCP Relay → API	No (transient)	Request lifetime
API responses	External service	MCP Relay → client	No (transient)	Request lifetime
Invocation logs	MCP Relay	Logging platform	Yes	30 days
API credentials	Secrets manager	MCP Relay (runtime)	In-memory only	Process lifetime

Data the server does NOT handle

- User passwords or personal credentials
- Persistent user data or profiles
- File uploads or binary content (not planned initially)
- Cached API responses (stateless architecture)

---

5. Authentication & Authorisation

User Authentication (IdP Integration)

- **Protocol:** OAuth 2.1 (MCP specification requirement for HTTP transport)
- **Flow:** Authorization Code with PKCE
- **IdP:** Corporate identity provider (SSO)
- **What we need:** OAuth client registration — client ID, client secret, redirect URI, allowed scopes
- **User experience:** User opens MCP client → redirected to company SSO login → authenticates → returned to client with token
- **Token validation:** MCP Relay validates tokens against the IdP's JWKS endpoint on every request
- **Session model:** Stateless — no server-side sessions; token validity checked per-request

Why OAuth 2.1 / SSO?

- **MCP specification mandate:** The protocol requires OAuth 2.1 for HTTP transport — this is not optional
- **No individual API tokens:** Users authenticate with existing company credentials; they never see or manage API tokens for Jira, S3, etc.
- **Centralised access control:** User access is governed by the IdP — disabling a user's SSO account immediately revokes MCP Relay access
- **Audit trail:** Every tool invocation is tied to an authenticated user identity

Two Authentication Layers

MCP Relay requires two separate authentication mechanisms, serving different purposes:

Layer	Purpose	Protocol
Corporate IdP (SSO)	Verify the user is allowed to use MCP Relay	OAuth 2.1 (PKCE)
Per-service user auth	Act as the user when calling external services	Service-specific OAuth

Corporate IdP controls access to MCP Relay itself. This is the IdP registration request this document supports.

Per-service user auth enables user-attributed actions in external services. For example, Jira supports OAuth 2.0 (3LO — three-legged OAuth), which allows MCP Relay to act on behalf of each user. When Steve updates an issue through MCP Relay, Jira records "Steve updated this issue" — not a generic service account. Each user authorises MCP Relay once per service ("MCP Relay would like to access your Jira account"), and MCP Relay stores a per-user refresh token to maintain access.

Without per-service user auth, all actions would appear as a shared service account, losing individual attribution and audit trail within the external service.

> **Security note:** Per-user refresh tokens are stored in the secrets manager alongside other credentials. Token scope follows the principle of least privilege — only the permissions required for the tools MCP Relay exposes.

Authorisation (Current & Future)

Stage	Model
Initial	All authenticated users can access all tools (flat access); per-user attribution via service-specific OAuth (e.g. Jira 3LO)
Future	Per-team or per-role tool access via scopes or group claims from IdP (RBAC)

---

6. Credential Management

MCP Relay centralises API credentials so that individual users never need direct access to service tokens.

- **Storage:** Secrets manager (e.g. AWS Secrets Manager) — never in code, config files, or logs
- **Access:** Injected at container startup; read into memory only
- **Rotation:** Supported without redeployment (secrets manager handles rotation)
- **Scope:** One set of credentials per external service, managed centrally
- **Audit:** Secrets manager logs who accessed which secret and when (e.g. CloudTrail)
- **Local development:** `.env` file (gitignored), validated at startup with Zod

Credentials inventory

Local development (service account)

Credential	Service	Permission Level
`JIRA_API_TOKEN`	Jira Cloud	Read/write issues
`JIRA_USER_EMAIL`	Jira Cloud	Service account email (shared identity for all API calls)
`JIRA_BASE_URL`	Jira Cloud	Instance URL
`AWS_ACCESS_KEY_ID`	AWS S3	Read-only access to config bucket
`AWS_SECRET_ACCESS_KEY`	AWS S3	Read-only access to config bucket
`S3_BUCKET_NAME`	AWS S3	Bucket identifier

Deployed (per-user attribution)

Credential	Service	Permission Level
Jira OAuth client ID + secret	Jira Cloud	Registered OAuth 2.0 (3LO) app with Atlassian
Per-user Jira refresh tokens	Jira Cloud	User-scoped; actions attributed to individual user
`JIRA_BASE_URL`	Jira Cloud	Instance URL
`AWS_ACCESS_KEY_ID`	AWS S3	Read-only access to config bucket
`AWS_SECRET_ACCESS_KEY`	AWS S3	Read-only access to config bucket
`S3_BUCKET_NAME`	AWS S3	Bucket identifier

> **Note:** The local development service account credentials (`JIRA_API_TOKEN` , `JIRA_USER_EMAIL`) are replaced by the Jira OAuth 2.0 (3LO) flow once deployed. S3 remains service-level as there is no per-user access requirement for brand guidelines.

---

7. Adding New Tool Integrations

MCP Relay is designed to grow its tool catalogue over time. New integrations mean new external service connections and potentially new API credentials. This section defines the governance process for adding tools.

Proposed Change Control Process

> **Note:** This process is a starting proposal.

Step	Action	Who
1	**Proposal** — document the service, permissions required, data accessed, and business case	MCP Relay team
2	**Security review** — assess data flow, credential scope, and risk	Security / InfoSec
3	**Approval** — sign-off on the new integration	TBD
4	**Credential provisioning** — create service account, store credentials in secrets manager	Service owner + DevOps
5	**Implementation** — build tool handler following established patterns	MCP Relay team
6	**Deployment** — release via CI/CD pipeline	MCP Relay team

What each new tool integration includes

- Zod-validated input and output schemas (no unvalidated data crosses boundaries)
- Documented data flow diagram (what goes where)
- Defined credential scope (principle of least privilege)
- Rate limiting configuration (cost and abuse prevention)
- Structured logging (audit trail for every invocation)

---

8. Audit & Logging

What is logged

Event	Fields
Tool invocation	Timestamp, user identity, tool name, input parameters (sanitised)
Tool result	Timestamp, tool name, outcome (success/error), latency
Authentication event	Timestamp, user identity, result (success/failure)
Rate limit hit	Timestamp, user identity, tool name
Server lifecycle	Startup, shutdown, configuration loaded

What is NOT logged

- API credentials or tokens (OWASP requirement)
- Full API response bodies (may contain sensitive data)
- User passwords or SSO tokens

Log infrastructure

- **Format:** Structured JSON
- **Destination:** Centralised logging platform (e.g. CloudWatch, Datadog)
- **Retention:** 30 days minimum
- **Access:** Restricted to MCP Relay team and authorised support staff

---

9. Risk Summary

Risk	Mitigation	Residual Risk
Unauthorised access to external services	OAuth 2.1 authentication required; IdP controls user lifecycle	Low
Credential exposure	Secrets manager; never in code, logs, or error messages; Zod validation prevents accidental leak	Low
Uncontrolled tool proliferation	Change control process for new integrations (Section 7)	Low
Excessive API usage / cost overrun	Per-tool rate limiting; structured logging for usage monitoring	Low
Data leakage via MCP responses	Response validation (Zod); only curated fields returned to clients	Low
Service account over-privileging	Principle of least privilege; credentials scoped per-service	Low
Server compromise	Stateless architecture; no persistent data; credentials in-memory only	Medium